```
"""
ARKHEIA-CPS — Analysis Router
GET  /analysis/{contract_id}   → Retrieve
stored analysis result
GET  /analysis                 → List all
analysis results (advocate/regulator)
POST /analysis/{contract_id}/rerun → Re-
run analysis on existing contract
"""
from fastapi import APIRouter, Depends,
HTTPException, status, Query
from sqlalchemy.orm import Session
from typing import List, Optional
from uuid import UUID

from app.db import get_db
from app.models.common import
AnalysisResult, Contract

router = APIRouter(prefix="/analysis",
tags=["Analysis"])


@router.get("/{contract_id}")
def get_analysis(contract_id: UUID, db:
Session = Depends(get_db)):
    """
    Retrieve the stored RTCSA analysis
result for a contract.
    Returns full scores, alerts,
```

```
explanations, PEM evaluations, and
statutes.
    """
    result =
db.query(AnalysisResult).filter(
        AnalysisResult.contract_id ==
contract_id
    ).first()

    if not result:
        raise HTTPException(

status_code=status.HTTP_404_NOT_FOUND,
            detail=f"No analysis found for
contract {contract_id}"
        )

    return _format_result(result)


@router.get("/")
def list_analyses(
    vertical:   Optional[str] = Query(None,
description="Filter by AUTO or HOUSING"),
    risk_level: Optional[str] = Query(None,
description="Filter by GREEN, YELLOW,
RED"),
    limit:      int = Query(50, ge=1,
le=200),
    offset:     int = Query(0, ge=0),
    db:         Session = Depends(get_db),
):
    """
    List all stored analysis results.
```

```
    Supports filtering by vertical and risk
level.
    Useful for advocate/regulator
dashboards.
    """
    query = db.query(AnalysisResult)

    if vertical:
        query =
query.filter(AnalysisResult.vertical ==
vertical.upper())
    if risk_level:
        query =
query.filter(AnalysisResult.risk_level ==
risk_level.upper())

    total   = query.count()
    results =
query.order_by(AnalysisResult.created_at.de
sc()).offset(offset).limit(limit).all()

    return {
        "total":   total,
        "offset":  offset,
        "limit":   limit,
        "results": [_format_result(r) for r
in results],
    }


@router.get("/summary/stats")
def analysis_stats(db: Session =
Depends(get_db)):
    """
```

```
    Aggregate statistics across all
analyzed contracts.
    Used for advocate/regulator reporting
dashboards.
    """
    from sqlalchemy import func

    total     =
db.query(AnalysisResult).count()
    red_count =
db.query(AnalysisResult).filter(AnalysisRes
ult.risk_level == "RED").count()
    yel_count =
db.query(AnalysisResult).filter(AnalysisRes
ult.risk_level == "YELLOW").count()
    grn_count =
db.query(AnalysisResult).filter(AnalysisRes
ult.risk_level == "GREEN").count()
    avg_score =
db.query(func.avg(AnalysisResult.overall_ri
sk_score)).scalar()
    auto_count =
db.query(AnalysisResult).filter(AnalysisRes
ult.vertical == "AUTO").count()
    house_count =
db.query(AnalysisResult).filter(AnalysisRes
ult.vertical == "HOUSING").count()

    return {
        "total_contracts_analyzed": total,
        "by_risk_level": {
            "red":    red_count,
            "yellow": yel_count,
            "green":  grn_count,
```

```
        },
        "by_vertical": {
            "auto":    auto_count,
            "housing": house_count,
        },
        "average_risk_score":
round(float(avg_score or 0), 2),
    }


def _format_result(r: AnalysisResult) ->
dict:
    """Serialize an AnalysisResult ORM
object to a clean API response dict."""
    dim_scores = {}

    if r.vertical == "AUTO" or
str(r.vertical) == "AUTO":
        dim_scores = {
            "affordability":
float(r.affordability_score or 0),
            "fee":
float(r.fee_score or 0),
            "term":
float(r.term_score or 0),
            "vehicle_safety":
float(r.vehicle_safety_score or 0),
            "enforcement":
float(r.enforcement_score or 0),
            "perfection":
float(r.perfection_risk_score or 0),
        }
    else:
        dim_scores = {
```

```
            "affordability":
float(r.affordability_score or 0),
            "fee":
float(r.fee_score or 0),
            "lease_term":
float(r.lease_term_score or 0),
            "habitability":
float(r.habitability_score or 0),
            "eviction_safety":
float(r.eviction_safety_score or 0),
        }

    return {
        "id":                   str(r.id),
        "contract_id":
str(r.contract_id),
        "vertical":
str(r.vertical),
        "overall_risk_score":
float(r.overall_risk_score),
        "risk_level":
r.risk_level,
        "dimension_scores":    dim_scores,
        "triggered_alerts":
r.triggered_alerts_json or [],
        "explanations":
r.explanations_json or [],
        "pem_evaluations":
r.pem_evaluations_json or [],
        "statutes":
r.statutes_json or [],
        "created_at":
```

```
str(r.created_at),
    }
```